Augmentation và transfer learning

Thêm biến thiên, dùng lại tri thức

Chương 5 · Bài 5.3

Sáu mươi ảnh chai, nhiều điều kiện chụp

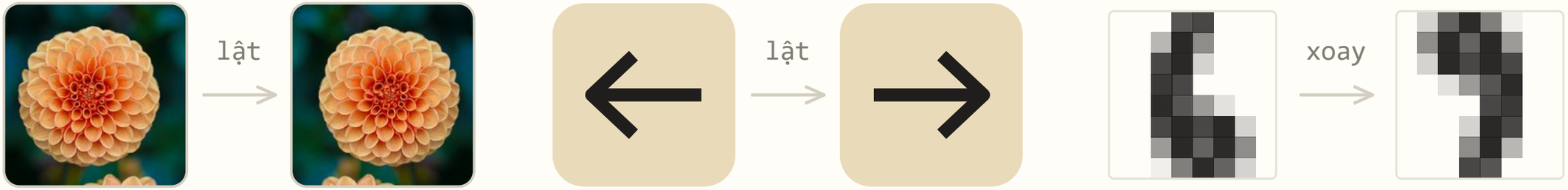
Phép biến đổi nào vẫn giữ nhãn “chai nước”?

Biểu diễn từ nguồn nào chuyển được sang ảnh mới?

01

Augmentation là một giả định

Lật ảnh có giữ đáp án?



hoa — **vẫn là hoa**

rẽ trái — **thành rẽ phải**

số 6 — **thành số 9**

nhãn giữ hay đổi tùy điều mà nhãn mô tả

Cùng một phép biến đổi, ba kết quả khác nhau: lật ảnh hoa vẫn là hoa; lật biển rẽ trái thành rẽ phải; xoay chữ số 6 (ảnh load_digits) thành hình số 9.

Hai hợp đồng khác nhau

Classification: .

Tọa độ, mask: .

HỎI NHANH

Bài toán là phân biệt chữ “b” với chữ “d”. Lật ngang ảnh mà vẫn giữ nhãn cũ thì có ổn không?

A Ổn, vì lật ngang an toàn với mọi bài phân loại ảnh

B Ổn, nếu mỗi epoch chỉ lật một nửa số ảnh

C Không ổn, vì chữ “b” lật ngang trông như chữ “d”

D Không ổn, vì phép lật chỉ dùng được cho ảnh màu

Ba nhóm phép biến đổi

Hình học: lật, crop, xoay, dời → áp cả lên box, mask, điểm mốc.

Cường độ: sáng, tương phản, màu, nhiễu → giữ vị trí, có thể đụng tín hiệu nhãn.

Che và trộn: Cutout giữ nhãn.

Mixup: ,

nhãn .

CutMix: trộn nhãn theo diện tích.

Năm phép trên một ảnh thật



gốc



lật ngang



crop rồi phóng



xoay 15°



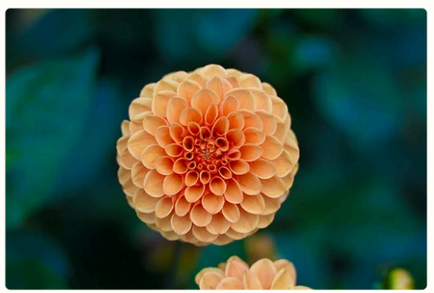
sáng hơn, nhạt hơn



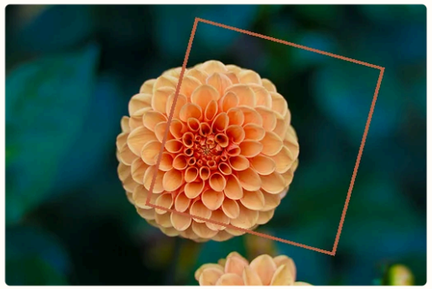
cutout

Năm phép augmentation thường gặp trên một ảnh thật: lật ngang, crop rồi phóng, xoay, đổi sáng và tương phản, cutout. Với nhãn “tháp”, cả năm đều giữ nhãn.

Mỗi epoch một phiên bản khác



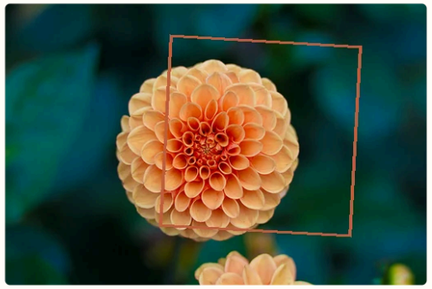

ảnh gốc




epoch 2: crop 48% diện tích, xoay +15°, lật ngang, sáng ×1.05




epoch 5: crop 60% diện tích, xoay −4°, lật ngang, sáng ×1.24




epoch 8: crop 46% diện tích, xoay +3°, sáng ×1.01

Mỗi epoch, cùng một ảnh hoa được crop, xoay, lật và đổi sáng ngẫu nhiên; nhãn vẫn là hoa.

HỎI NHANH

Model dự đoán tọa độ mắt trái và mắt phải trên ảnh chân dung rộng . Khi lật ngang ảnh, ta đã đổi mọi tọa độ thành . Nhãn còn thiếu bước nào?

A Đổi chỗ hai điểm, mắt trái thành mắt phải

B Không thiếu gì, vì tọa độ đã biến đổi theo ảnh

C Đổi thêm thành cho đối xứng

D Chia tọa độ cho để đưa về khoảng

Tính tay lật box

Tọa độ theo biên: vùng ngang là , không gồm pixel ở .

: .

Kiểm box và mask cùng biến đổi

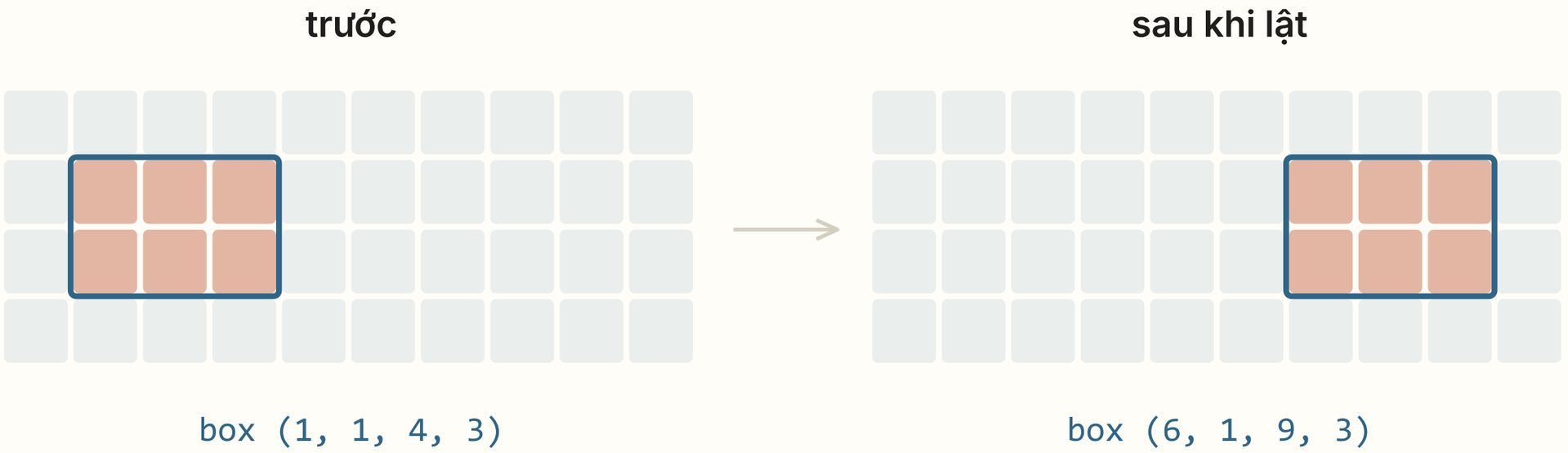
Trích khối code của bài: `mask` sáng ở hàng 1–2, cột 1–3. Đoạn dưới kiểm cả tọa độ lẫn số pixel foreground.

```
box = np.array([1, 1, 4, 3])
flipped_mask = np.flip(mask, axis=1).copy()
flipped_box = flip_box_xyxy(box, mask.shape[1])
print(flipped_box.tolist())
print(np.flatnonzero(flipped_mask[1]).tolist())
print(int(mask.sum()), int(flipped_mask.sum()))
```

KẾT QUẢ

```
[6, 1, 9, 3]
[6, 7, 8]
6 6
```

Mask và box lật cùng nhau



Lật ngang mask 4×10 và box xyxy cùng lúc (số từ code bài): mask sáng ở cột 1–3 chuyển sang cột 6–8, box (1, 1, 4, 3) thành (6, 1, 9, 3).

HỎI NHANH

Mask có ID lớp 0, 1 và 2. Resize bằng bilinear tạo ra giá trị 1.4 ở biên giữa vùng lớp 0 và vùng lớp 2. Như vậy có sao không?

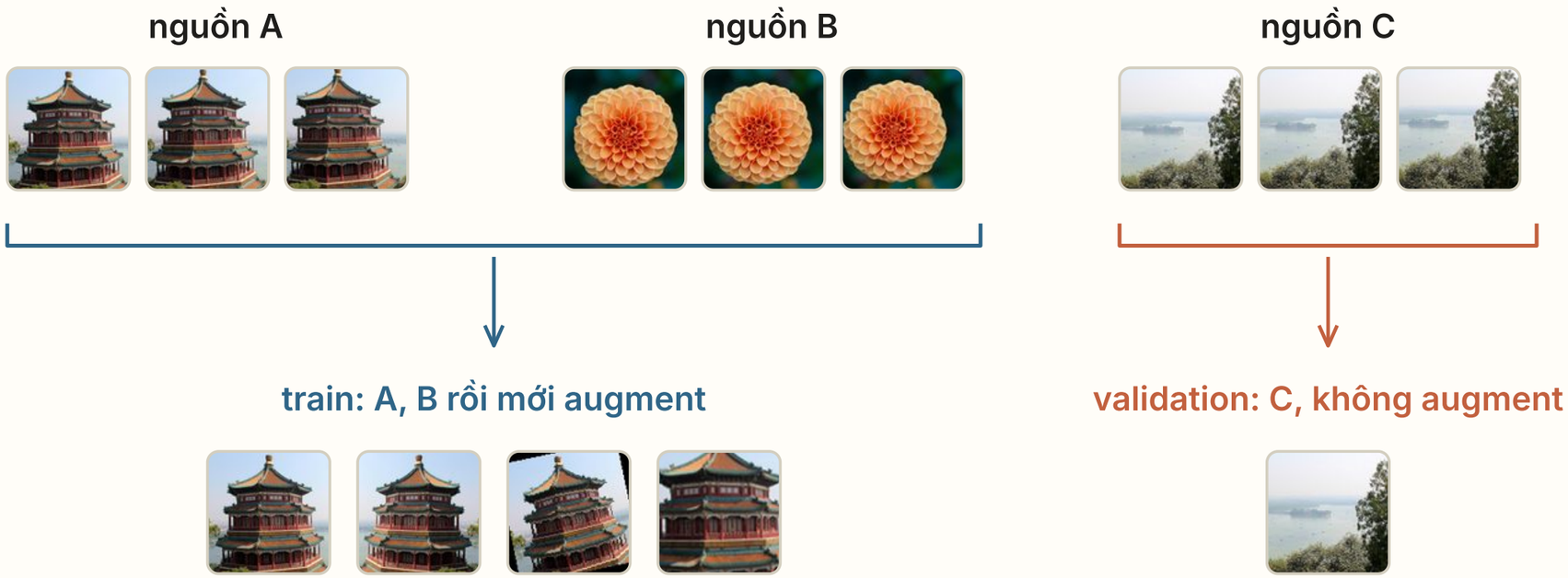
A Không sao, ép về `long` thì 1.4 thành lớp 1 hợp lệ

B Không sao, biên đã có thêm một lớp trung gian thật

C Có, phải chuẩn hóa mask về trước khi resize

D Có, ID lớp không nội suy được, nên phải dùng nearest

Chia theo nguồn trước



các frame của một nguồn gần như trùng nhau: chia theo nguồn, không chia theo ảnh

Chia theo nguồn trước khi augment: mọi frame của nguồn C nằm trọn trong validation; các biến thể augmentation chỉ sinh từ ảnh train.

Validation là ảnh sẽ gặp

Train: biến đổi ngẫu nhiên.

Validation: preprocessing xác định, chia theo nguồn.

TTA: trung bình dự đoán của vài phiên bản ảnh test.

Chỉ hợp lệ khi mọi phiên bản giữ nhãn; runtime nhân theo số phiên bản.

HỎI NHANH

Model phân loại biển báo “rẽ trái” và “rẽ phải”. Một đội thêm TTA bằng cách lấy trung bình xác suất của ảnh gốc và ảnh lật ngang. Điểm sẽ thay đổi ra sao?

A Điểm tăng, vì TTA luôn giảm dao động của dự đoán

B Xác suất hai lớp bị kéo về gần nhau, điểm giảm

C Điểm không đổi, vì lúc train model đã gặp ảnh lật

D Điểm giữ nguyên, chỉ thời gian suy luận tăng gấp đôi

02

Thí nghiệm: dời và lật chữ số

Bốn cấu hình, một split

300 ảnh chữ số để train.

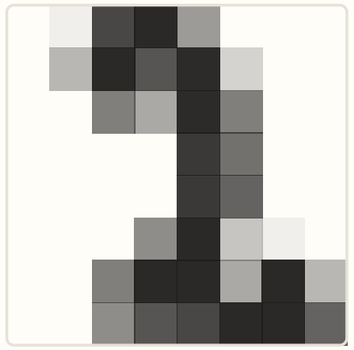
Validation hai bản: ảnh gốc, ảnh dời 1 pixel.

Dời 1 pixel: giữ nhãn.

Lật ngang: giữ nhãn cũ dù chữ đã khác.

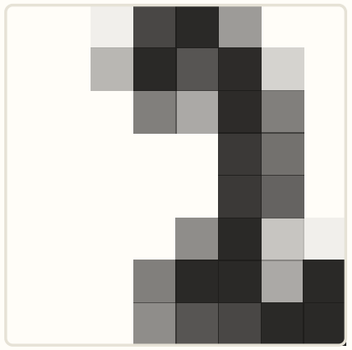
Trung bình 3 seed.

Nhìn ảnh trước khi train



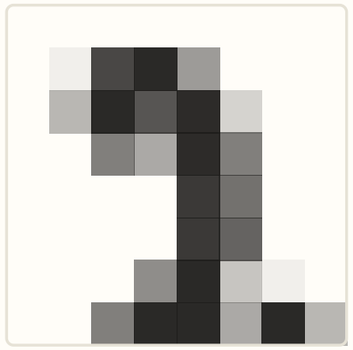
gốc

nhãn 2



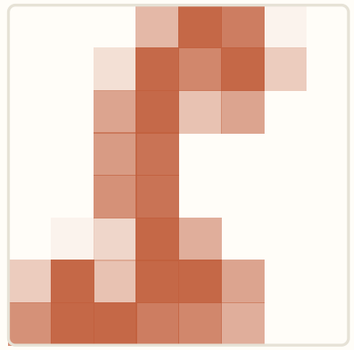
dời phải

nhãn 2



dời xuống

nhãn 2



lật ngang

nhãn 2

Một ảnh chữ 2 thật của load_digits: dời một pixel vẫn là chữ 2; lật ngang tạo một hình không còn là chữ 2 nhưng vẫn mang nhãn 2.

Đọc ablation

KẾT QUẢ

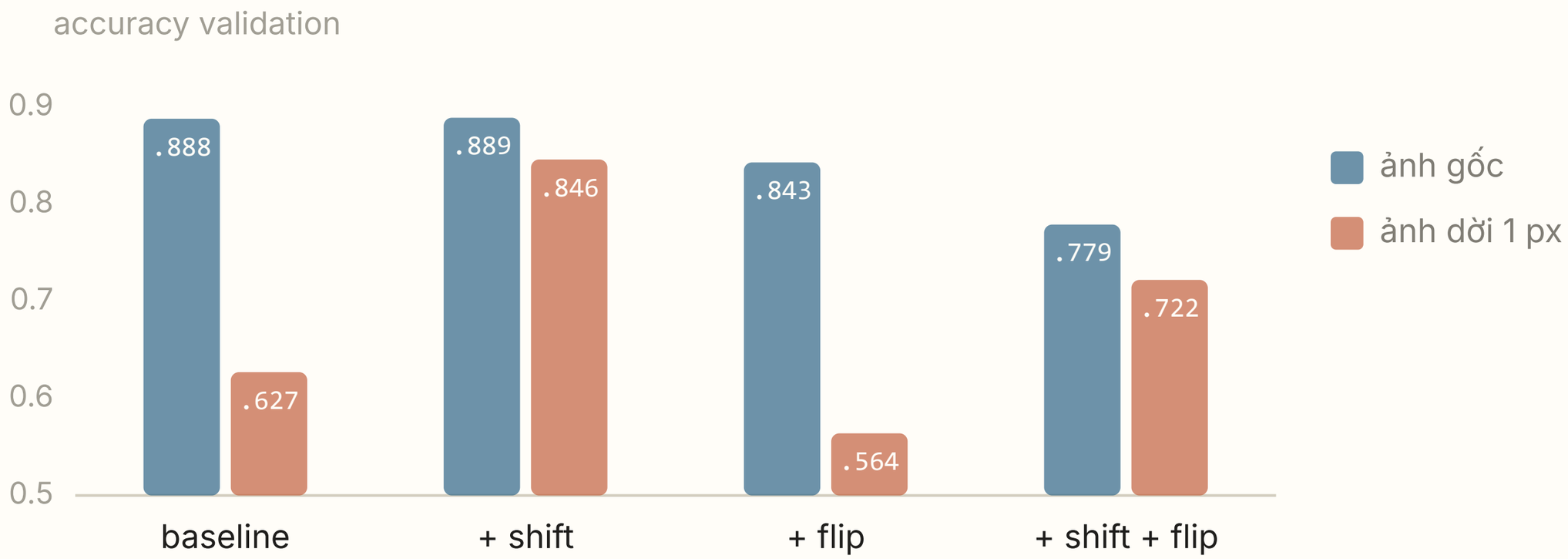
config	original	shifted
baseline	0.888	0.627
+ shift	0.889	0.846
+ flip	0.843	0.564
+ shift + flip	0.779	0.722

Dời: ảnh gốc không đổi rõ (trong dao động seed), ảnh dời 0.627 → 0.846.

Lật: giảm cả hai cột.

Ghép: phép lật ăn bớt lợi của phép dời.

Hai cột, bốn cấu hình



Ablation trên load_digits (số từ output bài, trung bình ba seed): phép dời gần như không đổi cột ảnh gốc nhưng nâng cột ảnh dời từ 0.627 lên 0.846; phép lật kéo cả hai cột xuống.

HỎI NHANH

Một đội chỉ có validation gồm ảnh gốc. Baseline được 0.888, thêm phép dời được 0.889. Họ bỏ phép dời vì cho rằng nó không có tác dụng. Bạn thấy kết luận ấy thế nào?

A Hợp lý, phép dời không đổi điểm thì chỉ tốn thời gian train

B Hợp lý, nên thay bằng lật ngang cho điểm thay đổi rõ hơn

C Chưa đủ, cần thêm seed tới khi cột ảnh gốc tăng rõ rệt

D Chưa đủ, validation không có ảnh dời để đo đúng tác dụng ấy

03

Chuyển biểu diễn

Encoder và head

Encoder tạo vector hoặc feature map.

Head học nhãn của nhiệm vụ hiện tại.

Đóng băng encoder: feature extraction.

Mở block cuối hoặc toàn mạng: fine-tuning.

Cơ chế freeze, BatchNorm, optimizer: bài 4.3.

Chọn điểm xuất phát

	Miền gần nguồn	Miền xa nguồn
Ít ảnh	Feature extraction	Block giữa, mở dần
Nhiều ảnh	Mở block cuối, rate nhỏ	Mở nhiều; so với từ đầu

Block đầu (cạnh, vân) dùng lại được; block sau gắn với vật thể nguồn.

HỎI NHANH

Có 600 ảnh siêu âm có nhãn và một ResNet ImageNet. Lấy đặc trưng ở block cuối rồi train head thì validation thấp hơn nhiều so với kỳ vọng. Nên thử gì tiếp?

A Dùng đặc trưng của block giữa, hoặc mở thêm block encoder

B Bỏ pretrained, train ResNet-50 từ đầu trên đúng 600 ảnh này

C Giữ nguyên encoder, train head lâu gấp mười lần

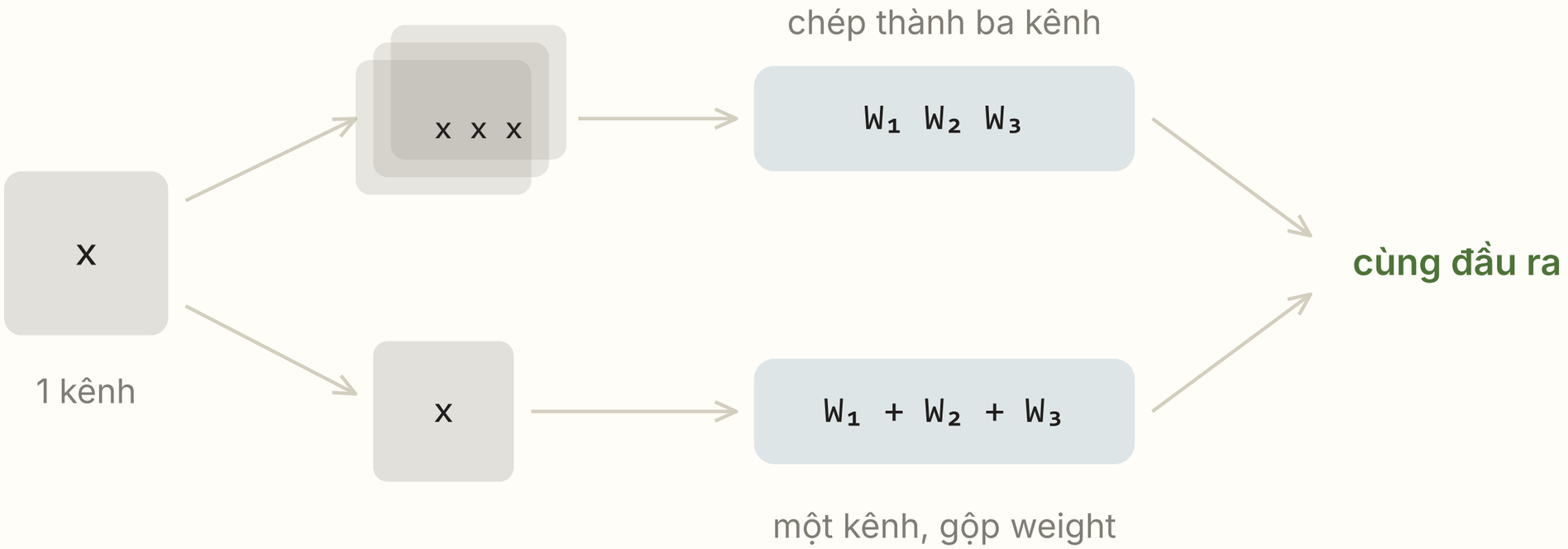
D Đổi sang encoder ImageNet lớn hơn, vẫn chỉ lấy block cuối

Ảnh xám vào encoder RGB

Chép kênh xám ba lần ≡ conv một kênh với weight **tổng**.

ResNet-18 conv đầu: số.

Chép kênh hay gộp weight



Ảnh xám vào conv RGB: chép kênh xám thành ba kênh cho cùng đầu ra với một conv một kênh có weight bằng tổng ba lát $W_1 + W_2 + W_3$.

Kiểm trên bốn ảnh chữ số

```
conv_gray.weight.copy_(
    conv_rgb.weight.sum(dim=1, keepdim=True))
out_repeat = conv_rgb(gray.repeat(1, 3, 1, 1))
out_folded = conv_gray(gray)
```

KẾT QUẢ

```
weight: (8, 3, 3, 3) -> (8, 1, 3, 3)
output: (4, 8, 8, 8) (4, 8, 8, 8)
same output: True
```

Hợp đồng đầu vào: số kênh, mean/std theo kênh, kích thước.

Ảnh còn ở block cuối; phóng lên 224 tốn gấp 64 lần.

HỎI NHANH

Để đưa ảnh xám vào ResNet ImageNet, một đội thay conv đầu bằng conv một kênh. Weight của nó là **trung bình** ba lát weight RGB. Mọi tầng sau giữ nguyên, và model được dùng ngay để lấy đặc trưng. So với cách chép kênh xám thành ba kênh, đầu ra của conv đầu khác thế nào?

A Giống hệt, vì trung bình và tổng chỉ khác tên gọi

B Nhỏ đi 3 lần, nhưng ReLU phía sau tự bù lại thang đo

C Phần trước bias nhỏ đi 3 lần, các tầng sau nhận sai thang

D Lớn lên 3 lần, vì bias cũng bị chia đều cho ba kênh

Pretrained là trọng số đã nạp

Recipe cục bộ: checkpoint + metadata (`sha256`, 1000 tên lớp) + ảnh.

Lớp đứng đầu hợp nghĩa: đường nạp và preprocessing khớp.

Chưa phải head của bài; `weights=None` là mạng ngẫu nhiên.

HỎI NHANH

Chạy recipe với một ảnh chai nước của câu lạc bộ, model in ra `water bottle 0.62`. Bài cần phân ba lớp là chai, cốc và hộp. Ta kết luận được gì?

A Model đã sẵn sàng, chỉ cần đọc ba lớp ImageNet gần nhất

B Đường nạp có vẻ đúng, nhưng vẫn cần head ba lớp và validation

C Trọng số chưa được nạp, vì model đã học sẽ cho xác suất trên 0.9

D Preprocessing sai, vì ảnh câu lạc bộ khác phân phối ImageNet

Khi đi thi

- Đọc luật asset: cấm pretrained hay được cấp checkpoint.
- Lưu grid ảnh sau augmentation kèm nhãn.
- Validation chia theo nguồn, mô phỏng điều kiện chấm.
- Feature extraction làm mốc, rồi mới fine-tune.
- TTA nhân thời gian forward: đo trước khi chọn.

04

Luyện đề

LUYỆN ĐỀ 1

X-quang ngực, dấu hiệu bệnh là phổi sáng hơn. Ảnh test lệch nhiều nhất 10°, encoder là ResNet ImageNet. Bước train nào có thể làm sai nhãn?

A `Grayscale(3)` chép một kênh thành ba

B `RandomRotation(10)`

C `ColorJitter(0.3, 0.3)`

D `Normalize` theo ImageNet

PHỎNG THEO NOAI 2026 Singapore · Chung kết · Câu 12

LUYỆN ĐỀ 2

X-quang xám. A chia cho 255 được 82.3%. B chuẩn hóa theo toàn tập được 84.7%. C chuẩn hóa từng ảnh theo mean, std của chính nó được 79.1%, loss dao động. Vì sao C kém nhất?

A Đầu vào không có trung bình 0, BatchNorm mất ổn định

B Nó xóa khác biệt độ sáng giữa các phim

C Nó nén dải giá trị, nên mất tương phản

D Nó rò thống kê validation vào train

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 307

LUYỆN ĐỀ 3

Mỗi ảnh có một hình tròn, tam giác hoặc chữ nhật, với vị trí, cỡ, góc và màu bất kỳ. Bộ augmentation nào giữ đúng nhãn?

A Xoay, lật, dời, đổi màu hình

B Giãn trục ngang 2:1, đổi màu

C Random crop từ ảnh

D Cutout một ô , giữ nhãn cũ

PHỎNG THEO USA-NA-AIO 2026 · Vòng 2 · Ngày 2 · Problem 4

Chốt ý

Augmentation là giả định: nhãn phải còn đúng sau biến đổi.

Đo trên validation có đúng điều kiện phép đó nhắm tới.

Tri thức nguồn cần đúng hợp đồng đầu vào và được kiểm trên miền đích.

Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook; thêm một phép augmentation mới, đoán trước hai cột `original` và `shifted` đổi thế nào rồi kiểm lại.